Combine Functions into Class

- When loading an XML File, a lot happens in the background.
- The XML File must be parsed correctly to properly generate Objects such as Exam, Chapter, Subtask, Variant
- Instead of implementing the Code into the “hello-view” the code should be written in a class called “XMLParser”.

- Also, there happens a lot in the background when an Exam is generated.
- Valid Combinations of subtasks from the selected Exam Type must be randomly picked and correctly displayed in a PDF File. Also, if there are multiple variants in a subtask, one of them should be randomly picked.
- Instead of writing the Code in the “PdfviewerController” the code should be written in a class called “ExamGenerator”
- This makes sense to divide UI-Code from logical code and to keep a clear structure.
- The screenshot below shows the parsing-function, which is part of XMLParser.java

```java
public class XMLParser {  new *

    public Exam parse(InputStream inputStream) throws Exception {  1 usage  new *

        DocumentBuilderFactory factory = DocumentBuilderFactory.newInstance();
        DocumentBuilder builder = factory.newDocumentBuilder();

        Document document = builder.parse(inputStream);
        document.getDocumentElement().normalize();

        Element examElement = document.getDocumentElement();

        Exam exam = new Exam();

        exam.setId(examElement.getAttribute( name: "id"));
        exam.setTitle(getText(examElement,  tag: "title"));

        List<Chapter> chapters = new ArrayList<>();

        NodeList chapterNodes = document.getElementsByTagName("chapter");

        for (int i = 0; i < chapterNodes.getLength(); i++) {
            Element chapterElement = (Element) chapterNodes.item(i);
            Chapter chapter = parseChapter(chapterElement);
            chapters.add(chapter);
        }

        exam.setChapters(chapters);

        return exam;
    }
```

Extract Function

- In the Class “ExamGenerator” large methods are divided into smaller functions to improve the structure.
- The original generation logic was previously implemented in one larger method and was then split into smaller methods such as “addAllSubtasks” and “isIncluded”.
- Repetitive logic is extracted into reusable methods to reduce code duplication.
- Each function now has only one specific task, for example to find the valid subtask combinations of a chapter or to find a random variant of a subtask.
- This is important to improve the structure, especially in the method “generatePracticeExam”, where multiple responsibilities were previously combined
- The screenshots below show some parts of how the generation method of a practice exam are divided into multiple ones:

-
```java
private static List<GeneratedTask> generatePracticeExam(Exam exam) {  1 usage  new *
    List<GeneratedTask> result = new ArrayList<>();

    for (Chapter chapter : exam.getChapters()) {
        if (!isIncluded(chapter)) {
            continue;
        }

        if (!DataValidator.isChapterValid(chapter, ExamType.PRACTICE)) {
            return new ArrayList<>();
        }

        addAllSubtasks(result, chapter, ExamType.PRACTICE, DifficultyLevel.EASY);
        addAllSubtasks(result, chapter, ExamType.PRACTICE, DifficultyLevel.MEDIUM);
        addAllSubtasks(result, chapter, ExamType.PRACTICE, DifficultyLevel.HARD);
    }

    return result;
}
```

```java
private static boolean isIncluded(Chapter chapter) {  2 usages  new *
    return chapter != null && chapter.getExamAppearance() == ExamAppearance.INCLUDE;
}
```

```java
private static void addAllSubtasks(List<GeneratedTask> result,  3 usages  new *
                                   Chapter chapter,
                                   ExamType examType,
                                   DifficultyLevel difficultyLevel) {
    List<Subtask> subtasks = DataValidator.getValidSubtasksByDifficulty(chapter, examType, difficultyLevel);

    for (Subtask subtask : subtasks) {
        Variant randomVariant = getRandomUsableVariant(subtask);

        if (randomVariant != null) {
            result.add(new GeneratedTask(chapter, subtask, randomVariant));
        }
    }
}
```

Move Function

- Instead of writing all methods in the controllers of the screens, they are moved to the best matching classes
- This makes sense because validation logic belongs to a dedicated validation class rather than UI controllers.
- For example, the function “isChapterValid” was written in the class “DataValidator.java” and not in a completely different one.
- The screenshot below shows how it is implemented

- ```java
  public class DataValidator {  new *

      public static boolean isChapterValid(Chapter chapter, ExamType examType) {  7 usages  new *
          List<Subtask> easy = getValidSubtasksByDifficulty(chapter, examType, DifficultyLevel.EASY);
          List<Subtask> medium = getValidSubtasksByDifficulty(chapter, examType, DifficultyLevel.MEDIUM);
          List<Subtask> hard = getValidSubtasksByDifficulty(chapter, examType, DifficultyLevel.HARD);

          if (easy.isEmpty() || medium.isEmpty() || hard.isEmpty()) {
              return false;
          }

          return hasValidDistribution(easy.size(), medium.size(), hard.size());
      }
  ```
  ```